Nidaan

Consultation Management and Adverse Drug Event Detection

DBMS Project Report

Student name	[Enter name]
SRN	[Enter SRN]
Course / section	[Enter course and section]
Faculty / institution	[Enter faculty and institution]
Submission date	[Enter date]

> Replace all bracketed placeholders before submission.

1. Title

Nidaan: Consultation Management and Adverse Drug Event Detection

Nidaan is a clinic management platform that connects appointment booking, consultations, prescriptions, adverse-event reporting, pharmacovigilance analysis and invoice generation.

2. Students SRN and Name

#	Name	SRN	Section	Contribution
1	[Student 1]	[SRN]	[Section]	[Contribution]
2	[Student 2]	[SRN]	[Section]	[Contribution]

3. Abstract

Nidaan is a clinic consultation and adverse drug event detection platform. It stores patients, appointments, consultations, prescriptions, drugs, interactions, invoices and safety reports in PostgreSQL. Database constraints preserve integrity, transactions protect concurrent booking and billing, and triggers automatically create interaction alerts and reminders. Adverse-event narratives are embedded with a 384-dimensional language model vector and searched with pgvector, allowing semantically similar reports to be retrieved even when patients use different words. The system supports receptionist, prescriber, pharmacovigilance and administrator roles through a Next.js interface and server-side authorization.

4. Tech Stack

Layer	Technology	Purpose
Frontend	Next.js App Router, React, TypeScript	Role-specific application pages and forms.
API	Next.js route handlers	Protected server-side operations.
Database	PostgreSQL	Relational clinical, appointment and billing data.
Vector database feature	pgvector	384-dimensional embeddings and cosine similarity.
Database driver	node-postgres (`pg`)	Connection pooling and parameterized queries.

Authentication	bcrypt and signed httpOnly cookies	Password hashing, session creation and role guards.
Embedding model	Xenova/all-MiniLM-L6-v2	Converts adverse-event text into vectors.
Tools	Git, GitHub, PowerShell, psql	Version control, development and database testing.

5. SQL — Database Commands

5.1 Schema and tables

The schema begins by enabling pgvector and creates the relational tables: `user_account`, `doctor`, `patient`, `appointment`, `consultation`, `drug`, `prescription`, `prescription_item`, `drug_interaction`, `interaction_alert`, `ade_report`, `invoice`, `invoice_line`, `reminder_queue`, `audit_log` and `patient_allergy`.

```
CREATE EXTENSION IF NOT EXISTS vector;
psql -d nidaan -f db/01_schema.sql
psql -d nidaan -f db/02_triggers.sql
psql -d nidaan -f db/03_procedures.sql
psql -d nidaan -f db/04_views.sql
psql -d nidaan -f db/05_seed.sql
```

5.2 Constraints and integrity

- Primary keys identify every row; foreign keys connect the clinical chain.
- Role, appointment mode, status, positive quantity and non-negative price use CHECK constraints.
- Online appointments require a meeting link; in-person appointments cannot have one.
- `drug_a_id < drug_b_id` prevents reverse duplicate interaction pairs.
- `ON DELETE RESTRICT` protects historical clinical data.

5.3 Required SQL evidence

Evidence	Command / result to include	Status
Tables	`\dt` output	[Insert screenshot]
Constraints	`\d appointment` and `\d drug_interaction`	[Insert screenshot]
Triggers	Interaction, reminder and audit trigger demonstrations	[Insert screenshot]
Procedures	Invoice and severity function output	[Insert screenshot]
Views	View or materialized-view query output	[Insert screenshot]

Indexes	`\di` output, including HNSW and GIN indexes	[Insert screenshot]

Insert SQL command/output screenshots here

6. NoSQL / Vector Database Commands

Nidaan uses PostgreSQL with the pgvector extension rather than a separate NoSQL server. The vector component is appropriate for free-text narratives, while the relational component preserves clinical integrity.

6.1 Embedding generation

```
npm run embed:seed
```

This embeds every adverse-event report whose `narrative_vec` is null. The model returns a normalized 384-dimensional vector.

6.2 Full-text baseline

```
SELECT report_id, narrative
FROM ade_report
WHERE narrative_tsv @@ plainto_tsquery('english', 'bleeding gums');
```

6.3 Vector similarity

```
SELECT report_id, narrative,
       1 - (narrative_vec <=> $1) AS similarity
FROM ade_report
WHERE narrative_vec IS NOT NULL
ORDER BY narrative_vec <=> $1
LIMIT 5;
```

The vector query can retrieve semantically similar wording such as “blood while brushing teeth, unusual bruising” for a report mentioning “bleeding gums and purple patches on arms”.

Insert vector query and similarity-result screenshot here

7. Next.js — Project Tree Structure

```
nidaan/
├── app/
│   ├── (auth)/login/
│   ├── reception/
│   ├── doctor/
│   ├── pharmacovigilance/
│   ├── admin/
│   └── api/
├── db/
│   ├── 01_schema.sql
│   ├── 02_triggers.sql
│   ├── 03_procedures.sql
│   ├── 04_views.sql
│   └── 05_seed.sql
├── lib/
│   ├── db.ts
│   ├── auth.ts
│   └── embeddings.ts
├── scripts/
├── docs/
├── package.json
└── README.md
```

Pages in `app` render the interface. Route handlers under `app/api` perform protected database operations. The `lib` folder contains shared database, authentication and embedding logic.

8. Sample Screenshots of Application Code and Demo

Pair each application screenshot with a short code excerpt and a one-sentence explanation.

Figure	Recommended screenshot	Explanation
8.1	Login page and login route	bcrypt verifies the password and a signed session cookie is created.
8.2	Receptionist booking screen	A receptionist selects a patient, doctor, time and appointment mode.

8.3	Doctor consultation screen	The doctor records complaint, notes and diagnosis.
8.4	Prescription builder with interaction alert	The trigger detects a dangerous drug pair.
8.5	Pharmacovigilance similarity result	Vector search retrieves differently worded narratives.
8.6	Admin dashboard	Admin counts, drug catalogue, interactions and audit logs are protected by role.
8.7	403 authorization demonstration	A disallowed role is rejected by the API.

Insert application screenshots and code excerpts here

> Do not include passwords, DATABASE_URL, SESSION_SECRET or .env.local in screenshots.

9. Evaluation Rubrics — Completion Status

Rubric area	Completion evidence	Status
Relational schema	Tables, PKs, FKs, UNIQUE and CHECK constraints	[Completed / Pending]
Advanced SQL	Triggers, stored functions, views and indexes	[Completed / Pending]
Concurrent booking	Exactly one conflicting booking succeeds	[Test result]
Drug interaction detection	`interaction_alert` with severity	[Test result]
Reminder queue	Patient and doctor reminder rows	[Test result]
Vector component	Embeddings, cosine search and similarity result	[Test result]
Security	bcrypt, sessions, parameterized queries and role guards	[Completed / Pending]

Frontend	Four role-specific workflows	[Completed / Pending]
Testing	Actual outputs and screenshots in `docs/test-results.md`	[Completed / Pending]
Documentation	ER diagram, relational schema, report and screenshots	[Completed / Pending]

Conclusion

Nidaan combines relational correctness with vector-based semantic discovery. PostgreSQL protects the clinical workflow, triggers and functions automate database behaviour, and pgvector helps the pharmacovigilance role discover adverse-event patterns beyond exact keyword matching.